

Backtranslating Fluent APIs

26M30498 | Dennis Prüne

PRG Lab

Contents

- Background
- Related Work
- My Proposal
- Current Progress & Summary

Background

What are Fluent APIs?

```
List<Integer> ys = new ArrayList();  
for (int x : xs) {  
    if (x >= 0) {  
        ys.add(2 * x);  
    }  
}
```

Imperative Style

- For-loops, if-clauses, ...
- Nested

```
List<Integer> ys = xs  
    .stream()  
    .filter(x -> x >= 0)  
    .map(x -> 2 * x)  
    .collect();
```

Fluent Style

- Method chaining
- Flat

Background

Fluent APIs 🤝 Embedded DSLs

```
SELECT name, birth_year  
FROM Students  
WHERE birth_year == 2005
```

naïve embedding

```
Query query = SQL.query(  
    "SELECT name, birth_year "  
    + "FROM Students "  
    + "WHERE birth_year == 2005");
```

fluent embedding

```
Query query = SQL  
    .select("name, birth_year")  
    .from("Students")  
    .where("birth_year == 2005");
```

Background

The Tradeoff

Advantages

- Less syntax to remember
- Dynamic data support

```
SQL.select(field).from("Students")
```

can be set
programmatically



Disadvantages

- NO static safety guarantees
 - NO syntax checking
 - NO type checking
 - NO semantic analysis

```
SQL.select("name, birth_year")  
  .from("Students")  
  .from("Teachers")
```

illegal syntax
causes runtime error



Related Work

Safe Fluent APIs[EriLex, H. Xu, 2010][Fling, Gil & Roth, 2019][...]

Goal: Disallow bad method chains at compile time

1. Encode allowed chains as a CFG

```
Query ::= SQL.select(String selections)  
                .from(String source)  
                Where
```

```
Where ::=  $\epsilon$  | .where(String clause)
```

2. Generate a fluent API where this syntax is encoded in the type system

Related Work

Limitations

- Only ensures partial syntactic correctness
- Verbose error messages

`SQL.select("1 + 6").from("Students")`

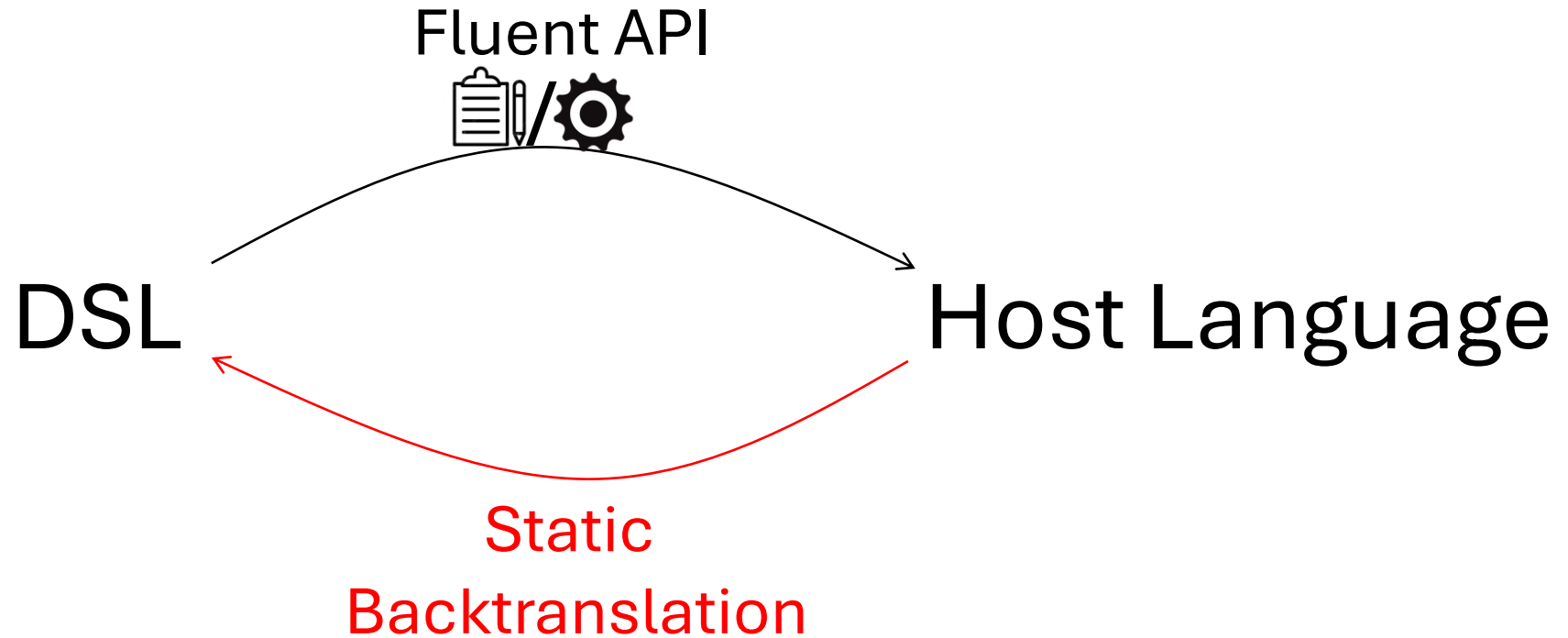
syntax error which cannot
be detected by safe fluent API

realistic error message

```
Test.hs:5:8:
  No instance for (Show r0) arising from a use of 'print'
  The type variable 'r0' is ambiguous
  Note: there are several potential instances:
    instance Show a => Show (Maybe a) -- Defined in 'GHC.Show'
    instance Show Ordering -- Defined in 'GHC.Show'
    instance Show Integer -- Defined in 'GHC.Show'
    ...plus 30 others
  In the expression: print
  -- omit 28 lines --
Test.hs:9:34:
  No instance for (ShapeTransition
    (Node5 (Node9 (Node6 (Node10 (Node11 (Node1 ()))))) s0)
    arising from a use of 'shape'
  In the second argument of '(|>)', namely 'shape "dotted"'
  In the first argument of '(|>)', namely
    'begin |> digraph "small_graph" |> node "A" |> shape "rectangle"
    |> node "B"
    |> node "C"
    |> shape "doublecircle"
    |> edge "A"
    |> to "B"
    |> shape "dotted"'
  In the first argument of '(|>)', namely
  -- omit 9 lines --
Test.hs:10:12:
  No instance for (EdgeTransition s0 s1) arising from a use of 'edge'
  The type variables 's0', 's1' are ambiguous
  -- omit the rest and three other errors --
```

Proposal

Why not translate back?



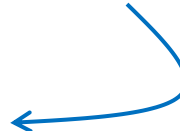
Proposal

Possible Use Cases

- Full syntax checking
- Semantic Analysis
- In-editor error messages
- Custom syntax highlighting

```
SQL.select("name, birth_year")  
  .from("Students")  
  .where("age == 16 AND name == 'Joe'")
```

this is how fluent API
code might look in the editor



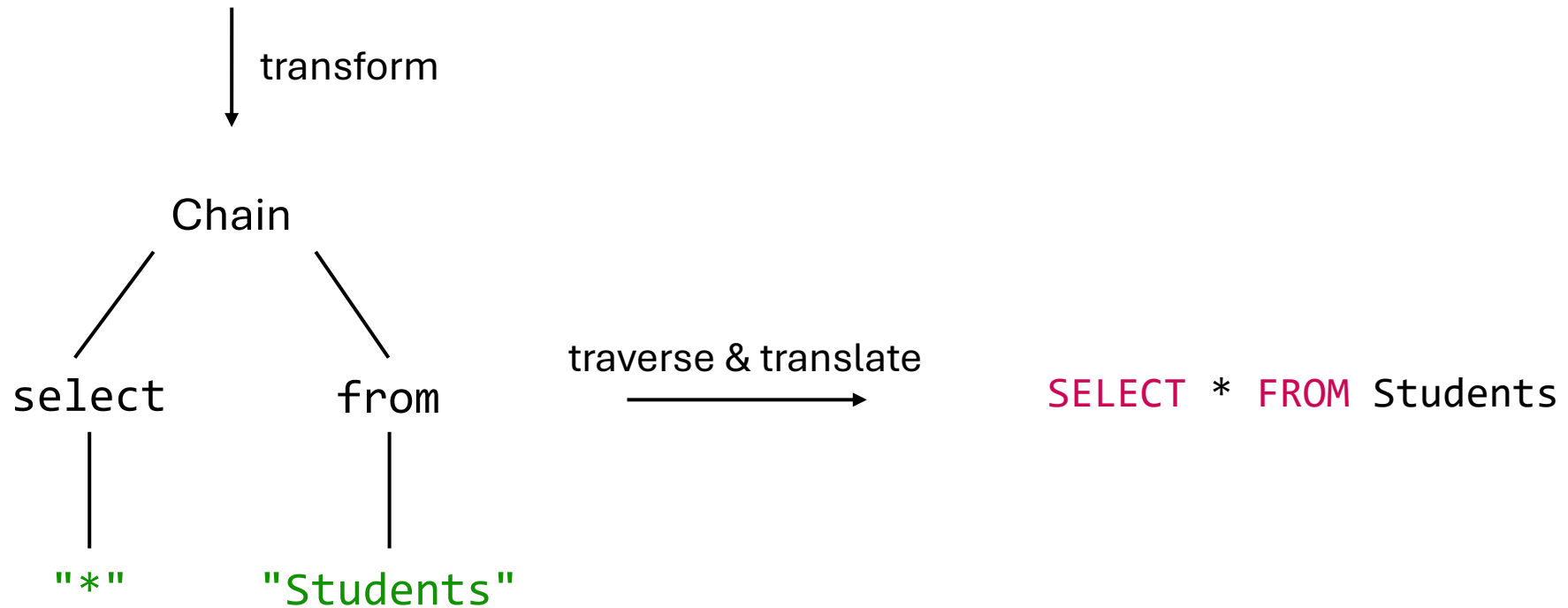
Unselected column 'age'

[Select 'age' from 'Students'](#)

Proposal

How can backtranslation be done?

```
SQL.select("*").from("Students")
```



Current Progress

- ✓ Come up with new method
- ☐ Create a Prototype
- ☐ Test it on existing Code

Summary

- Fluent APIs for embedded DSLs are very useful!
- ...static analysis is very limited
- I proposed a simple and extendable way to translate method chains back to the original DSL